SKYLITE / SKYNET – Developer Onboarding Guide

Table of Contents

1. Introduction
2. What Developers Must Understand First
3. Environment Setup
4. Project Structure Overview
5. How Routing Works
6. Understanding the WebPage Model
7. GET Lifecycle (Developer Perspective)
8. POST Lifecycle (Developer Perspective)
9. Working with Templates (HTML/TXT)
10. Working with HtmlDocument
11. Working with ApiRequest / ApiResponse
12. JSCore Basics
13. Translation System
14. Naming Conventions
15. Module Organization
16. Debugging & Diagnostics
17. Common Development Patterns
18. Building Forms
19. Building Grids
20. File Upload/Download
21. Security Hooks
22. Best Practices

23. Common Mistakes to Avoid

24. Appendix A – ASCII Diagrams

25. Appendix B – Example Pages

26. Appendix C – Developer Checklist

1. Introduction

This onboarding guide is designed to bring new developers up to speed with SKYLITE/SKYNET as quickly and effectively as possible. It explains the framework's architecture, development workflow, naming conventions, and lifecycle events in a practical, developer-friendly way.

The goal is to ensure that any developer can begin building pages confidently within their first day.

2. What Developers Must Understand First

Before writing any code, a developer must understand three core principles:

2.1 Server-Driven Rendering

The server generates the full HTML document. The browser only executes commands.

2.2 Reflection-Based Routing

URLs map directly to class names. No routing tables.

2.3 Deterministic Lifecycle

Every page follows the same GET/POST lifecycle.

If a developer understands these three ideas, everything else becomes intuitive.

3. Environment Setup

3.1 SKYLITE (IIS)

- Requires .NET Framework
- Runs under IIS
- Uses SkyLiteHandler

3.2 SKYNET (ASP.NET Core)

- Requires .NET 6+

- Runs cross-platform
- Uses IHandler middleware

3.3 Required Packages

- Microsoft.Data.SqlClient
- System.Drawing.Common

3.4 Required Configuration

```
"hotReloadEnabled": false
```

4. Project Structure Overview

A typical SKYNET project:

```
/Pages
    HR_UserProfile.cs
    CRM_CustomerList.cs
/Templates
    HR_UserProfile.html
    CRM_CustomerList.html
/wwwroot
    js/
    css/
Program.cs
appsettings.json
```

Important: **Folder structure does not affect routing.**

5. How Routing Works

```
/HR_UserProfile → HR_UserProfile : WebPage
```

5.1 ASCII Diagram

```
URL
  |
  v
ClassName
  |
  v
Reflection Lookup
  |
  v
WebPage Instance
```

5.2 Naming Rules

- Class names must be unique
- Namespaces may be used
- Recommended: Module_PageName

6. Understanding the WebPage Model

Every page inherits from:

```
public abstract class WebPage
```

6.1 Responsibilities

- Handle GET lifecycle
- Handle POST methods
- Manage HtmlDocument
- Integrate translation
- Provide security hooks

7. GET Lifecycle (Developer Perspective)

7.1 Sequence Diagram

```
OnLoad()
    ↓
OnInitialized()
    ↓
OnBeforeRender()
    ↓
OnAfterRender()
    ↓
HTML Output
```

7.2 What Developers Do in Each Stage

OnLoad

- Load template
- Initialize state

OnInitialized

- Add dynamic content
- Apply security rules

OnBeforeRender

- Final adjustments
- Prepare scripts

OnAfterRender

- Last-minute changes

8. POST Lifecycle (Developer Perspective)

8.1 Sequence Diagram

```
ApiRequest
    ↓
Resolve Page
    ↓
Invoke Method
    ↓
ApiResponse
    ↓
JSCore Executes
```

8.2 Developer Responsibilities

- Write methods returning ApiResponse
- Validate input
- Update database
- Return UI updates

9. Working with Templates (HTML/TXT)

9.1 Template Loading

Framework automatically loads:

```
PageName.html
```

9.2 Template Guidelines

- Keep layout simple
- Use IDs for fields
- Avoid inline scripts

10. Working with HtmlDocument

10.1 Structure

```
HtmlBodyText
HtmlBodyAddOn
HeadContent
ScriptInjection
```

10.2 Adding Dynamic Content

```
HtmlBodyAddOn += "<div>Dynamic</div>";
```

11. Working with ApiRequest / ApiResponse

11.1 ApiRequest

Sent by JSCore.

11.2 ApiResponse

Returned by server.

11.3 Common Commands

```
SetValue
UpdateHtml
ShowDialog
Notify
Redirect
RefreshGrid
```

RefreshGrid

12. JSCore Basics

JSCore handles:

- Sending ApiRequest
- Receiving ApiResponse
- Executing commands

- Updating DOM

Developers rarely need to modify JSCore.

13. Translation System

13.1 GET Translation

Applied to HtmlDocument.

13.2 POST Translation

Applied to ApiResponse.

13.3 Language Source

- Browser language
- Internal dictionary

14. Naming Conventions

14.1 Recommended Pattern

```
Module_PageName
```

14.2 Examples

```
HR_UserProfile
CRM_CustomerList
FIN_InvoiceEdit
```

15. Module Organization

Modules are logical, not physical.

15.1 Example Structure

```
HR_
CRM_
FIN_
INV_
```

16. Debugging & Diagnostics

16.1 Common Debugging Points

- Page resolution
- Template loading
- ApiResponse serialization
- JSCore command execution

16.2 Logging

Framework logs:

- Exceptions
- Request metadata
- Page/method names

17. Common Development Patterns

17.1 Form Submission

- Bind fields
- Validate
- Save
- Return ApiResponse

17.2 Grid Refresh

- Query database
- Return HTML
- Use UpdateHtml

18. Building Forms

18.1 HTML Structure

Use IDs for fields.

18.2 Server-Side Binding

Use SetValue.

18.3 Validation

Perform in POST method.

19. Building Grids

19.1 Server-Driven

Grid HTML is generated on the server.

19.2 Refresh Pattern

```
return ApiResponse.UpdateHtml("grid", html);
```

20. File Upload/Download

20.1 Upload Flow

- JSCore sends file
- Framework receives stream
- Developer processes

20.2 Download Flow

- Developer returns file stream
- Framework sets headers

21. Security Hooks

21.1 Hooks

- OnInitialized
- OnRequest
- OnBeforeRender

21.2 Usage

Check permissions early.

22. Best Practices

- Keep pages small
- Use naming prefixes
- Avoid heavy logic in OnLoad

- Use ApiResponse consistently
- Keep templates clean

23. Common Mistakes to Avoid

- Duplicate class names
- Overusing HtmlBodyAddOn
- Mixing business logic in lifecycle
- Returning malformed ApiResponse
- Forgetting translation tokens

24. Appendix A – ASCII Diagrams

24.1 GET Lifecycle

```
OnLoad → OnInitialized → OnBeforeRender → OnAfterRender
```

24.2 POST Lifecycle

```
Resolve Page → Invoke Method → ApiResponse → JSCore
```

25. Appendix B – Example Pages

25.1 Basic Page

```
public class HR_UserProfile : WebPage
{
    public override void OnLoad()
    {
        // Load template
    }

    public override void OnInitialized()
    {
        // Add dynamic content
    }

    public ApiResponse Save()
    {
        return ApiResponse.Success("Saved");
    }
}
```

26. Developer Checklist

Before writing code

- Understand routing
- Understand lifecycle
- Understand ApiResponse

When creating a page

- Use correct naming
- Create template file
- Implement lifecycle methods

When handling POST

- Validate input
- Return ApiResponse
- Update UI